Nantes Université

POLYTECH NANTES

ETN 5 – Polytech Nantes

Langages de description matérielle et réutilisation

Rapport de mini-projet

Réutilisation de codes HDL et HDL réutilisable

WU Riluke & LIU Chengpeng

Enseignant : Sébastien Pillement

Octobre 2026

Table des matières

Introduction **3**

2 Récupération d'un modèle de RAM **4**
2.1 Analyse du modèle et du testbench . . . 4
2.2 Passage au type unsigned et nouvelle simulation . . . 5
2.3 Paramétrage de la capacité et de la largeur des données . . . 6
2.4 Ajout d'une sélection asynchrone CS . . . 7

3 Réutilisation du bloc Sync_Ram **8**
3.1 Décodeur d'adresses . . . 8
3.2 Assemblage de quatre RAM . . . 8
3.3 Validation de l'ensemble . . . 9
3.4 Généricité du décodeur et de quadram . . . 10

4 Prise en main d'un modèle de processeur **12**
4.1 Valider le processeur RISC-V . . . 12
4.2 Manipuler les instructions RISC-V (§ 4.1.1 du sujet) . . . 13
4.3 Adapter et implémenter les instructions RISC-V (§ 4.1.2 du sujet) . . . 14
4.3.1 Modes d'adressage . . . 14
4.3.2 Corrections du modèle fourni . . . 15
4.3.3 Ajout des instructions de l'extension M . . . 17
4.3.4 Implémentation et validation de l'extension M . . . 18
4.4 Adapter la micro-architecture du processeur RISC-V (§ 4.2 du sujet) . . . 20

5 Création du système **23**
5.1 Accès aux données dans la mémoire externe . . . 23
5.2 Synchronisation avec le bloc quadram . . . 24
5.3 Micro-architecture du processeur . . . 25
5.4 Bloc système en VHDL . . . 25
5.5 Validation . . . 27
5.6 Différences avec le premier programme . . . 29

Conclusion **30**

Table des figures **31**

Liste des tableaux **32**

Bibliographie **33**

Introduction

Le sujet propose de construire un système à processeur en réutilisant une RAM décrite en VHDL et un processeur RISC-V décrit en Verilog [1]. Nous sommes partis de ces deux modèles pour obtenir un système dans lequel le programme reste dans le processeur, tandis que les données sont stockées dans une mémoire externe composée de plusieurs RAM.

Nous avons d'abord analysé la RAM, adapté ses types et rendu ses dimensions configurables. Une entrée de sélection et un décodeur permettent ensuite de partager le bus entre les instances. Côté processeur, nous avons corrigé les accès mémoire et le traitement des immédiats, ajouté les instructions de l'extension M, puis introduit une horloge et les signaux nécessaires à la connexion avec la mémoire.

Les simulations ont été réalisées sous ModelSim SE-64 2020.4. Nous présentons les modifications avec les chronogrammes, les valeurs attendues et les résultats des tests. Le rapport suit les parties 2 à 5 du sujet : RAM simple, bloc `quadram`, processeur et système complet. Les possibilités de synthèse sur FPGA et d'utilisation d'un compilateur RISC-V sont discutées en conclusion.

2 Récupération d'un modèle de RAM

Le premier bloc à réutiliser est une RAM synchrone fournie avec son testbench. Nous avons conservé son fonctionnement, puis adapté les types, les dimensions et la sélection pour pouvoir relier plusieurs instances au même bus.

2.1 Analyse du modèle et du testbench

Le modèle initial, `ramchip.vhd`, décrit l'entité `sync_ram`. Son adresse de 10 bits donne accès à $2^{10} = 1024$ mots de 8 bits. Les entrées sont l'horloge `clock`, l'autorisation d'écriture `we`, l'adresse et la donnée à écrire ; la sortie `dataout` fournit la donnée lue. Le tableau mémoire n'est pas initialisé.

Le code 1 distingue deux mécanismes. Au front montant, si `we = '1'`, la donnée est écrite à l'adresse présente sur l'entrée. Le même processus mémorise l'adresse dans `read_address`, quelle que soit la valeur de `we`. L'affectation concurrente de sortie utilise cette adresse mémorisée : un changement de l'adresse d'entrée ne suffit donc pas à actualiser la lecture, il faut attendre le front montant suivant.

Code 1 – Lecture et écriture du modèle initial

```
if rising_edge(clock) then
  if we = '1' then
    ram(to_integer(unsigned(address))) <= datain;
  end if;
  read_address <= address;
end if;
...
dataout <= ram(to_integer(unsigned(read_address)));
```

Le fichier `ram_tb.vhd` contient deux architectures : `Bench` présente un scénario court et `Bench2` effectue une boucle d'écriture suivie d'une boucle de lecture. Nous les avons simulées séparément. L'horloge a une période de 10 ns et s'arrête lorsque `StopClock` devient vrai.

Scénario court. Le testbench modifie ses entrées juste après un front montant, par des affectations de signaux. Le processus de la RAM voit encore les anciennes valeurs à ce front : les nouvelles valeurs seront prises en compte au front suivant. Le tableau 1 donne les principaux événements observés sur la figure 1.

Temps	Résultat observé
25 ns	Écriture de 4 au mot 1 ; la sortie prend la valeur 4 après mise à jour des signaux.
35 ns	Écriture de 7 au mot 2 ; la sortie prend la valeur 7.
45 ns	Le testbench demande le mot 1 ; l'adresse mémorisée reste 2 et la sortie reste 7.
55 ns	L'adresse 1 est mémorisée et la sortie prend la valeur 4.
65 ns	L'adresse 2 est mémorisée et la sortie prend la valeur 7.

TABLE 1 – Chronologie du test court de la RAM initiale

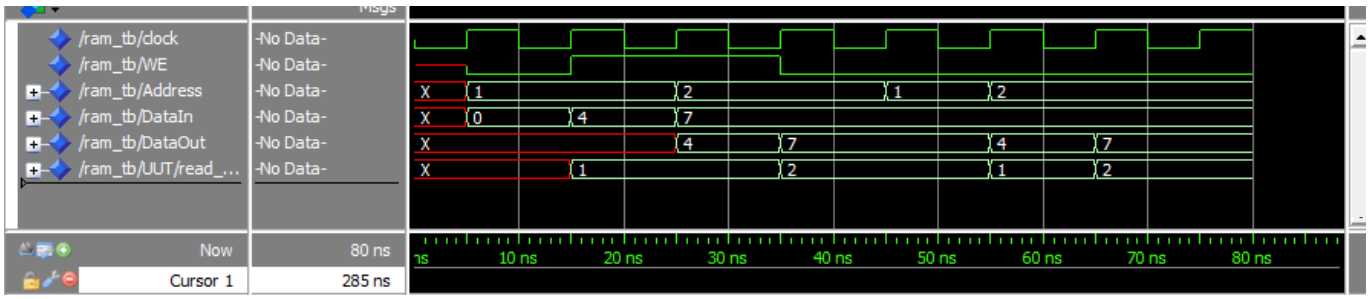


FIGURE 1 – RAM initiale : écritures de 4 et 7, puis lectures aux adresses 1 et 2

La sortie initiale indéfinie est attendue : ni la mémoire ni le registre d'adresse ne sont initialisés. ModelSim peut aussi signaler à 0 ns une métavaleur lors de la conversion de l'adresse en entier. Enfin, quand un front écrit un mot et mémorise en même temps son adresse, la sortie présente directement la nouvelle donnée : l'affectation concurrente de sortie est réévaluée après la mise à jour du mot et de l'adresse.

Test en boucle. `Bench2` utilise le signal booléen `ok`, qui reste faux dès qu'une comparaison échoue. Notre exécution se termine à 20490 ns avec `ok = TRUE`. La comparaison soustrait 1 à l'adresse courante car, au moment où elle est exécutée au front montant, `dataout` correspond encore à l'adresse précédente.

Le test a toutefois deux limites : les données se répètent tous les 256 mots, ce qui peut masquer un défaut sur les bits d'adresse de poids fort, et la dernière donnée lue au mot 1023 n'est pas comparée. Nous avons gardé ce test pour la comparaison avant/après, puis ajouté un contrôle de toutes les adresses à la section 2.3.

2.2 Passage au type unsigned et nouvelle simulation

Nous avons remplacé `std_logic_vector` par `unsigned` pour les ports d'adresse et de données, les mots mémoire, le registre d'adresse et les signaux du testbench. Ce type de `numeric_std` représente un entier non signé et permet d'écrire directement `address + 1`.

Les conversions entourant l'adresse deviennent inutiles : `to_integer(unsigned(address))` devient `to_integer(address)`. Le rôle des signaux, leurs largeurs et les conditions de lecture et d'écriture sont conservés. L'horloge et `we`, qui ne portent pas des nombres, restent de type `std_logic`.

Nous avons relancé les deux architectures après recompilation. Le scénario court reproduit les valeurs et les instants du tableau 1 (figure 2). Le test `Bench2` se termine à nouveau à 20490 ns, avec `ok = TRUE`. Ces deux observations confirment la conservation du comportement pour les scénarios testés.

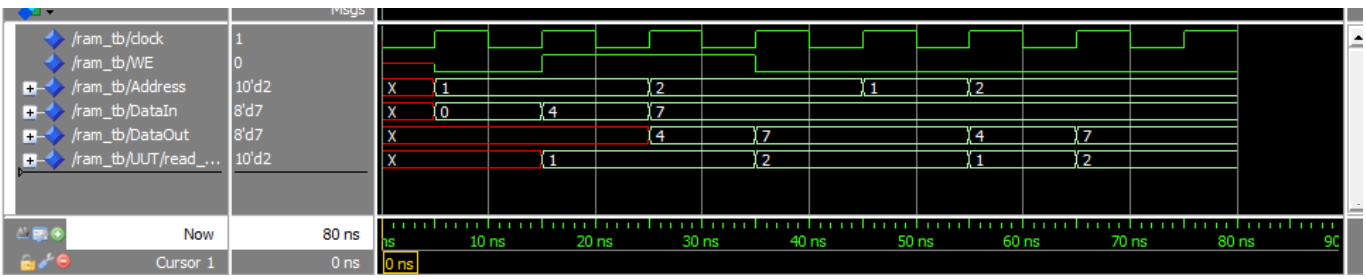


FIGURE 2 – RAM avec ports unsigned : même chronologie que le modèle initial

2.3 Paramétrage de la capacité et de la largeur des données

Deux paramètres `generic` déterminent les dimensions du bloc (code 2). `Nbits_addr` donne la largeur d'adresse, donc la capacité vaut $2^{\texttt{Nbits_addr}}$ mots ; `Nbits_data` donne le nombre de bits par mot. Nous les avons déclarés de type `positive` pour exclure les largeurs nulles. Les valeurs par défaut, 10 et 8, retrouvent le modèle initial.

Code 2 – Paramètres et ports de la RAM générique

```
generic (
  Nbits_addr : positive := 10;
  Nbits_data : positive := 8
);
port (
  clock   : in  std_logic;
  we      : in  std_logic;
  address : in  unsigned(Nbits_addr-1 downto 0);
  datain  : in  unsigned(Nbits_data-1 downto 0);
  dataout : out unsigned(Nbits_data-1 downto 0)
);
```

L'architecture utilise déjà les attributs des ports : le tableau contient `2**address'length` éléments, chacun de type `unsigned(datain'range)`, et `read_address` reprend `address'range`. Elle s'adapte ainsi aux dimensions choisies sans changer le processus de lecture et d'écriture. Un `generic map` permet de choisir une configuration propre à chaque instance.

Vérification de deux configurations. Le `Bench2` hérité compare encore les données à `address(7 downto 0) - 1` : il n'est pas adapté à toutes les nouvelles dimensions. Nous avons donc ajouté l'architecture `GenericCheck` dans `ram_generic_check.vhd`, en conservant les deux tests précédents.

Ce test écrit puis relit toutes les adresses, y compris la dernière. Les entrées changent au front descendant, la RAM les échantillonne au front montant et la lecture est contrôlée au front descendant suivant. Le motif de données replie les bits d'adresse par XOR sur la largeur du mot ; une seconde passe utilise son complément. Chaque bit de données est ainsi exercé à 0 et à 1, et tous les bits d'adresse participent au motif. Les deux configurations du tableau 2 passent ce contrôle.

Bits d'adresse	Bits de données	Mots	Fin de simulation	Résultat
4	16	16	660 ns	`ok = TRUE`
10	32	1024	40980 ns	`ok = TRUE`

TABLE 2 – Configurations vérifiées avec deux passes complémentaires

Le modèle de RAM et le testbench sont identiques pour les deux essais ; seuls les paramètres changent au chargement dans ModelSim. Le contrôle se termine respectivement à 650 et 40970 ns, puis l'horloge s'arrête une période plus tard. La figure 3 montre les deux passes pour la configuration 1024 mots de 32 bits.

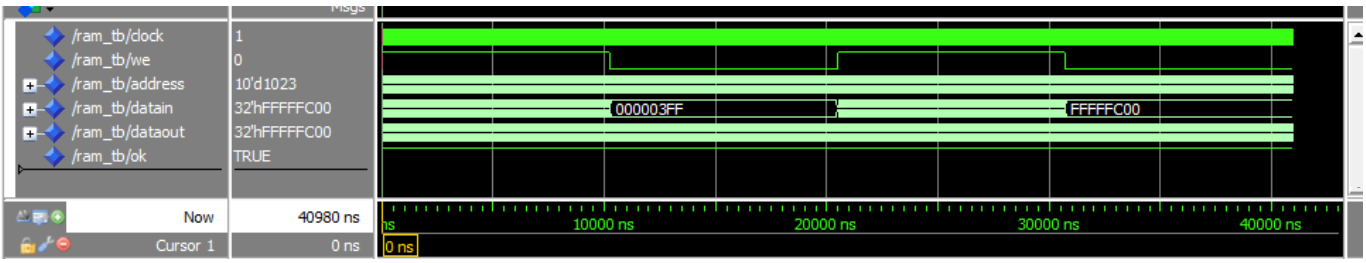


FIGURE 3 – Configuration 1024 mots de 32 bits : même testbench et résultat TRUE

2.4 Ajout d'une sélection asynchrone CS

Pour relier plusieurs sorties mémoire, une seule RAM doit piloter le bus à un instant donné. Nous avons ajouté l'entrée `cs`, active à 1. Une affectation concurrente impose la haute impédance lorsque la RAM n'est pas sélectionnée (code 3). La réaction à `cs` est asynchrone : elle n'attend pas de front d'horloge. L'adresse de lecture reste, elle, mémorisée au front montant.

Code 3 – Sélection des écritures et commande de la sortie

```
if rising_edge(clock) then
  if we = '1' and cs = '1' then
    ram(to_integer(address)) <= datain;
  end if;
  read_address <= address;
end if;
...
dataout <= ram(to_integer(read_address))
           when cs = '1' else (others => 'Z');
```

La sélection doit aussi conditionner l'écriture : avec un `we` commun, les RAM non sélectionnées seraient sinon modifiées. Les essais de la RAM avec sélection et de `quadram` ont été relancés après la correction décrite en section 5.5.

Le scénario court conserve d'abord les écritures de 4 et 7, puis désactive `cs` à 67 ns, entre deux fronts d'horloge. La sortie passe immédiatement en haute impédance. Au front montant de 85 ns, `we` vaut 1 et la donnée proposée est 9 à l'adresse 1, mais `cs` vaut 0 : le mot reste égal à 4. À 87 ns, la sélection est rétablie, toujours entre deux fronts, et la sortie reprend immédiatement la valeur 4 (figure 4). La trace interne `ram(1)` confirme la conservation du mot.

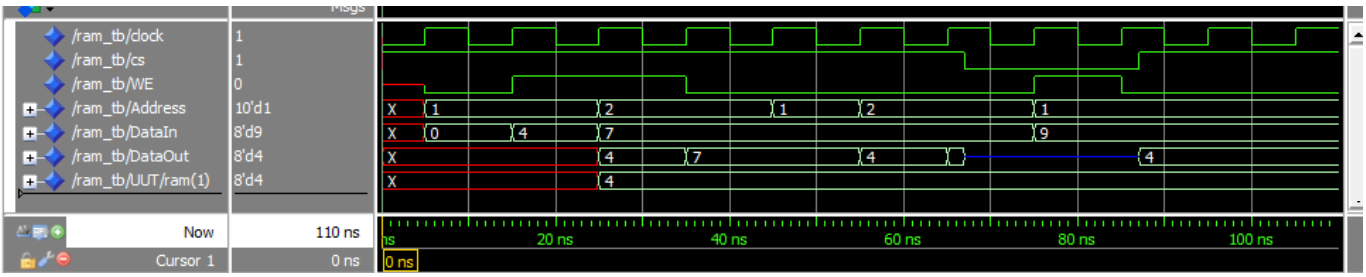


FIGURE 4 – CS asynchrone : sortie à haute impédance et écriture bloquée entre 67 et 87 ns

3 Réutilisation du bloc Sync_Ram

Nous avons ensuite construit `quadram`, un bloc qui réunit quatre instances de `Sync_Ram` derrière un décodeur. Les RAM partagent les entrées et le bus de sortie ; le décodeur doit donc sélectionner une seule instance à partir de l'adresse globale.

3.1 Décodeur d'adresses

Pour quatre RAM, deux bits de sélection suffisent. Nous avons utilisé une affectation `with ... select` pour produire le code à un seul bit actif du tableau 3. Le décodeur est combinatoire : il n'a ni horloge ni état interne.

Entrée `data_in`	Sortie `cs_out(3 downto 0)`	RAM sélectionnée
`00`	`0001`	RAM 0
`01`	`0010`	RAM 1
`10`	`0100`	RAM 2
`11`	`1000`	RAM 3
autres valeurs	`0000`	aucune

TABLE 3 – Décodage des deux bits de sélection

Le cas `when others` est nécessaire car les bits peuvent notamment prendre les valeurs indéfinies `U` ou `X`. Une adresse indéterminée ne doit pas activer une RAM. Le testbench applique successivement `00`, `01`, `10`, `11`, puis `XX`, pendant 10 ns chacun, et vérifie les sorties par assertions. Le chronogramme (figure 5) reproduit le tableau ; après 40 ns, l'entrée `XX` donne bien `0000`. Les vérifications se terminent à 50 ns.

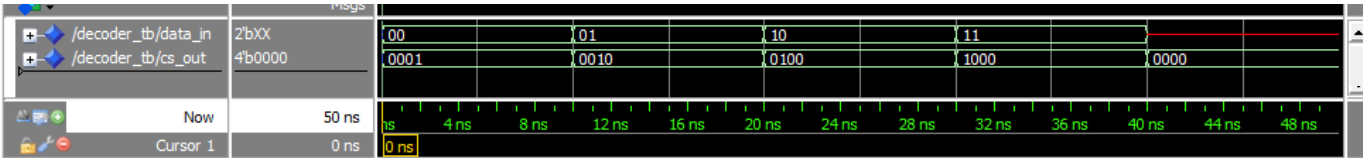


FIGURE 5 – Décodeur 2 vers 4 : une sélection par adresse valide et aucune sélection pour XX

3.2 Assemblage de quatre RAM

Le fichier `quadram.vhd` instancie le décodeur et quatre exemplaires du modèle `sync_ram` avec sélection. Chaque RAM est configurée avec `Nbits_addr = 8` et `Nbits_data = 8`, soit 256 mots de 8 bits. L'ensemble contient donc $4 \times 256 = 1024$ mots de 8 bits.

L'adresse globale comporte 10 bits. Les bits 9 et 8 sont appliqués au décodeur ; les bits 7 à 0 désignent le mot à l'intérieur de la RAM. L'adresse locale, les données à écrire, `we` et l'horloge sont communes aux

quatre instances. Seul le signal `cs(i)` est propre à chaque RAM. Le tableau 4 donne la répartition de l'espace d'adressage.

RAM	Bits 9–8	Adresses globales (hexadécimal)
0	`00`	`000` à `0FF`
1	`01`	`100` à `1FF`
2	`10`	`200` à `2FF`
3	`11`	`300` à `3FF`

TABLE 4 – Répartition des adresses entre les quatre RAM de 256 mots

Les sorties sont reliées au même signal `dataout`. Dans la simulation VHDL, les éléments résolus de type `std_logic` qui composent les vecteurs `unsigned` permettent cette connexion à plusieurs pilotes. La RAM sélectionnée fournit la donnée, les autres fournissent `Z`. Deux pilotes imposant des valeurs opposées produiraient une valeur `X`, d'où la nécessité de n'activer qu'une seule sortie.

3.3 Validation de l'ensemble

Nous avons compilé la RAM corrigée, le décodeur et `quadram` dans une même bibliothèque, puis exécuté `quadram_tb.vhd`. Le test écrit quatre valeurs différentes au déplacement local 5, puis relit les quatre adresses (tableau 5).

RAM	Adresse globale	Donnée écrite et relue	Sélection
0	`005`	`11`	`0001`
1	`105`	`22`	`0010`
2	`205`	`33`	`0100`
3	`305`	`44`	`1000`

TABLE 5 – Test des quatre RAM ; adresses et données en hexadécimal

La figure 6 montre les écritures de 10 à 50 ns puis les lectures de 50 à 90 ns. Les assertions ne signalent aucune erreur et le message « Fin du test de quadram » apparaît à 90 ns ; l'horloge s'arrête à 100 ns. Les quatre valeurs restent distinctes : une écriture dans une RAM n'a pas écrasé le mot de même déplacement dans les autres. Si l'écriture n'était pas conditionnée par `cs`, les quatre mots finiraient tous avec la dernière valeur, `44`, et les trois premières comparaisons échoueraient.

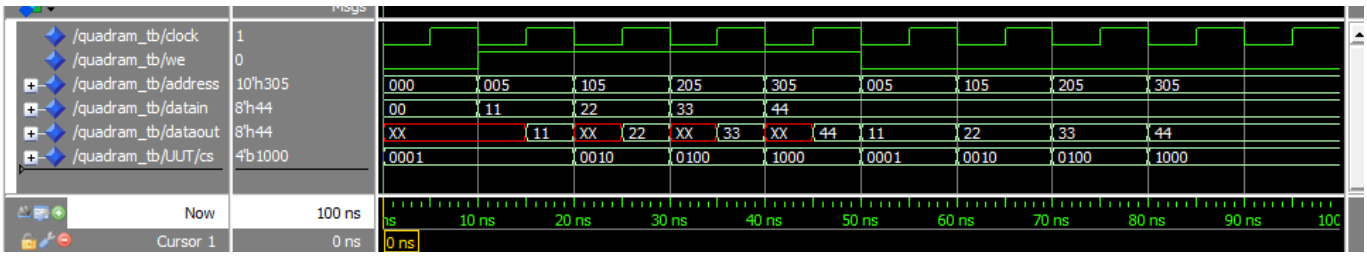


FIGURE 6 – Quadram fixe : écritures puis lectures indépendantes des quatre RAM

Interprétation du temps de lecture. Dans ce test, le déplacement local reste égal à 5 pendant le passage d'une RAM à l'autre. Le changement des bits de sélection modifie donc la sortie dès que le décodeur change `cs` : le mot local 5 est déjà adressé dans chaque RAM. Ce comportement ne signifie pas que toute lecture est asynchrone. Si le déplacement change aussi, la sortie peut transitoirement présenter la nouvelle RAM à l'ancienne adresse locale, jusqu'au front montant suivant. Le processeur devra attendre que cette adresse soit échantillonnée avant de capturer la donnée (section 5.2).

Ces quatre lectures vérifient la sélection et l'indépendance des blocs au déplacement 5 ; elles ne constituent pas un parcours de toutes les adresses de `quadram`.

3.4 Généricité du décodeur et de quadram

Nous avons rendu le décodeur générique avec `Nbits_sel`, le nombre de bits de sélection. Le décodeur produit $2^{\texttt{Nbits_sel}}$ sorties, chacune décrite par une affectation générée (code 4).

Code 4 – Décodage générique des signaux de sélection

```
GEN_CS: for i in 0 to 2**Nbits_sel-1 generate
  cs_out(i) <= '1' when data_in = i else '0';
end generate GEN_CS;
```

Le bloc `quadram` reçoit trois paramètres : `Nbits_sel`, `Nbits_addr` et `Nbits_data`, notés ici s, a et d. Il contient $N = 2^s$ RAM de 2^a mots de d bits, soit 2^{s+a} mots au total ; l'adresse globale a $s + a$ bits. Le nombre de RAM est donc une puissance de deux : cette variante ne couvre pas un nombre arbitraire de blocs.

Les instanciations explicites sont remplacées par le code 5. `generate` crée des instances matérielles parallèles à l'élaboration du modèle, et non une boucle d'exécution partageant une seule RAM.

Code 5 – Instances de RAM créées par for ... generate

```
RAMS: for i in 0 to 2**Nbits_sel-1 generate
  RAM: entity work.sync_ram(RTL)
    generic map (Nbits_addr => Nbits_addr, Nbits_data => Nbits_data)
    port map (cs => cs(i), clock => clock, we => we,
              address => address(Nbits_addr-1 downto 0),
              datain => datain, dataout => dataout);
end generate RAMS;
```

Validation de deux configurations. Le testbench écrit au déplacement 5 de la RAM i la valeur `0x11` multipliée par i + 1, puis vérifie la lecture par assertion. Le tableau 6 présente les deux configurations testées sans modification des sources.

Bits de sélection	Bits d'adresse locale	Bits de données	RAM	Mots par RAM	Résultat
3	4	16	8	16	aucune assertion en erreur
2	10	32	4	1024	aucune assertion en erreur

TABLE 6 – Configurations du quadram générique effectivement simulées

Avec huit RAM, les adresses `05`, `15`, jusqu'à `75` relisent `0011`, `0022`, jusqu'à `0088`. Le test se termine à 170 ns et l'horloge s'arrête à 180 ns (figure 7).

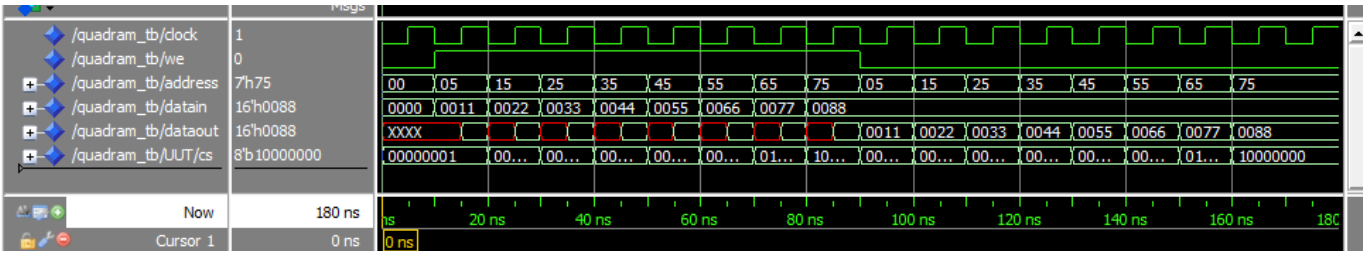


FIGURE 7 – Version générique : huit RAM de 16 mots de 16 bits, test des huit sélections

La configuration du système final contient quatre RAM de 1024 mots de 32 bits. L'adresse de 12 bits comprend deux bits de sélection et dix bits d'adresse locale. Les mots `005`, `405`, `805` et `C05` relisent respectivement `11`, `22`, `33` et `44` sur 32 bits. Les assertions passent, la simulation s'arrête à 100 ns (figure 8). La sélection et l'isolation sont ainsi vérifiées au déplacement utilisé dans les deux configurations.

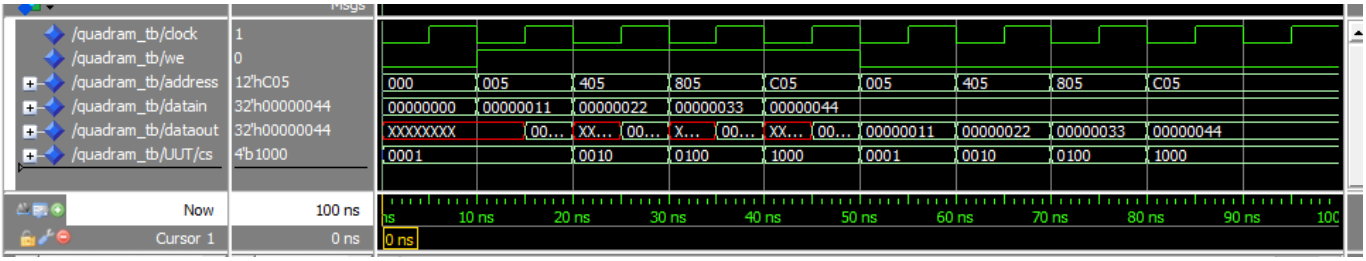


FIGURE 8 – Configuration du système final : quatre RAM de 1024 mots de 32 bits

4 Prise en main d'un modèle de processeur

Le processeur fourni sur Madoc (`riscvproc-2.v`) est un modèle comportemental Verilog qui implémente une partie du jeu d'instructions RV32I [2] : `add`, `sub`, `sll`, `srl`, `addi`, `andi`, `ori`, `lw`, `sw` et `ebreak`. Il contient une mémoire interne `memory` de 4096 mots de 32 bits, qui sert à la fois au programme et aux données, ainsi que les 32 registres `x0` à `x31`. Une instruction est exécutée toutes les `Step` = 10 ns ; le module `system` n'a aucun port, c'est un modèle purement simulable.

Le fichier utilise des constructions SystemVerilog (`localparam int unsigned`) : nous l'avons renommé avec l'extension `.sv` et compilé avec `vlog -sv`. L'instruction `ebreak` termine la simulation par `$finish` ; nous lançons donc `vsim` avec l'option `-onfinish stop`, qui arrête la simulation sans fermer le design et permet d'examiner ensuite les registres et la mémoire.

4.1 Valider le processeur RISC-V

Pour faire exécuter un programme, la mémoire du processeur est initialisée au début de la simulation à partir d'un fichier texte, avec la tâche système `$readmemh` placée dans le bloc `initial` du processeur (code 6). Le fichier contient un mot de 32 bits en hexadécimal par ligne ; `@0` donne l'adresse du premier mot et `//` introduit un commentaire. Le chemin est relatif au répertoire courant de ModelSim : si le fichier n'est pas trouvé, ModelSim l'indique par un avertissement, la mémoire reste à `X` et le processeur affiche « erreur : mauvaise valeur dans op-code » à chaque instruction.

Code 6 – Chargement du programme dans la mémoire du processeur

```
string progfile;
initial begin
  if (!$value$plusargs("prog=%s", progfile))
    progfile = "test_base.txt";
  $readmemh(progfile, memory);
  // initialize program counter to 0
  pc = 0;
  ...
end
```

Nous avons ajouté l'option `+prog` pour changer de programme sans recompiler le processeur. Sans cette option, le processeur du code 6 charge `test_base.txt` ; la commande `vsim work.system +prog=instr.txt` lui fait charger `instr.txt`.

Le programme de test fourni sur Madoc, `instr.txt`, contient six instructions suivies de deux données, 2 et 4, que la directive `@40` place aux mots 64 et 65 (40 en hexadécimal). Le tableau 7 le décode : à la fin du programme, on attend x1 = 1, x2 = 6 et la valeur 6 au mot 65.

Mot	Contenu	Instruction	Effet attendu
0	`04002083`	`lw x1, 64(x0)`	x1 = mot 64 = 2
1	`04102103`	`lw x2, 65(x0)`	x2 = mot 65 = 4
2	`00110133`	`add x2, x2, x1`	x2 = 6
3	`001150b3`	`srl x1, x2, x1`	x1 = 6 décalé de 2 bits = 1
4	`0420a023`	`sw x2, 64(x1)`	le mot 64 + x1 = 65 reçoit 6
5	`00100073`	`ebreak`	fin de la simulation
64	`00000002`	donnée	
65	`00000004`	donnée	

TABLE 7 – Programme `instr.txt`

Avec le modèle d'origine, aucun de ces résultats n'est obtenu (tableau 8 et figure 9). Les deux `lw` lisent le mot 0, c'est-à-dire la première instruction (`04002083`), au lieu des mots 64 et 65. L'addition et le décalage portent donc sur ce code : x2 = `08004106`, puis x1 = `01000820`, car le décalage vaut les 5 bits de poids faible de x1, soit 3. Enfin, `sw` écrit lui aussi au mot 0 : il écrase la première instruction du programme, et le mot 65 garde sa valeur initiale, 4. Cette erreur d'adresse, ainsi que celles révélées par le programme `test_base`, nous ont conduits aux corrections de la section 4.3.2. Après ces corrections, `instr.txt` donne les valeurs attendues et la simulation se termine à 70 ns.

	Attendu	Modèle d'origine	Après correction
x1	1	16779296 (`01000820`)	1
x2	6	134234374 (`08004106`)	6
mot 0	`04002083`	`08004106`	`04002083`
mot 65	6	4	6

TABLE 8 – Résultats de `instr.txt` avant et après correction du modèle

4.2 Manipuler les instructions RISC-V (§ 4.1.1 du sujet)

Les instructions RV32I et de l'extension M utilisées ici sont codées sur 32 bits. Le projet n'utilise pas l'extension C, qui introduit des instructions de 16 bits. Le modèle utilise trois formats (tableau 9) : le format R pour les opérations entre registres, le format I pour les opérations avec un immédiat de 12 bits et pour `lw`, et le format S pour `sw`, dont l'immédiat est découpé en deux champs afin que `rs1` et `rs2` restent à la même place que dans le format R.

```
# Break in Module system at riscvproc_original.sv line 153
VSIM 7> echo "=== AVANT correction (modele d'origine) ==="
# === AVANT correction (modele d'origine) ===
VSIM 8> examine -radix decimal /system/regs
# {0 16779296 134234374 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
VSIM 9> examine -radix hexadecimal {/system/memory[0]} {/system/memory[64]} {/system/memory[65]}
# 08004106 00000002 00000004
VSIM 10> quit -sim
ModelSim> vlog -quiet -sv riscvproc-2.sv
ModelSim> vsim -quiet -voptargs=+acc -onfinish stop work.system +prog=../../programmes/instr.txt
# End time: 17:46:22 on Oct 27,2026, Elapsed time: 0:00:14
# Errors: 0, Warnings: 0
# vsim -quiet -voptargs="+acc" -onfinish stop work.system "+prog=../../programmes/instr.txt"
# Start time: 17:46:22 on Oct 27,2026
# ** Note: (vsim-3813) Design is being optimized due to module recompilation...
VSIM 13> run -all
# erreur: mauvaise valeur dans op-code
# ** Note: $finish    : riscvproc-2.sv(153)
#    Time: 70 ns  Iteration: 0  Instance: /system
# Break in Module system at riscvproc-2.sv line 153
VSIM 14> echo "=== APRES correction ==="
# === APRES correction ===
VSIM 15> examine -radix decimal /system/regs
# {0 1 6 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
VSIM 16> examine -radix hexadecimal {/system/memory[0]} {/system/memory[64]} {/system/memory[65]}
# 04002083 00000002 00000006
```

FIGURE 9 – Transcript ModelSim : `instr.txt` avec le modèle d'origine, puis avec le modèle corrigé

	31 – 25	24 – 20	19 – 15	14 – 12	11 – 7	6 – 0
R	funct7	rs2	rs1	funct3	rd	opcode
I	`imm[11:0]`		rs1	funct3	rd	opcode
S	`imm[11:5]`	rs2	rs1	funct3	`imm[4:0]`	opcode

TABLE 9 – Formats d'instructions utilisés (spécification RISC-V, chapitre 2.2)

L'instruction `add x9, x20, x21` est de format R. Ses champs sont donnés dans le tableau 10 : le code obtenu est `0000 0001 0101 1010 0000 0100 1011 0011`, soit `0x015A04B3`. L'opcode `0110011` désigne les opérations entre registres ; `funct3` et `funct7` choisissent l'opération (`funct7` vaut `0100000` pour `sub`).

funct7	rs2 = x21	rs1 = x20	funct3	rd = x9	opcode
`0000000`	`10101`	`10100`	`000`	`01001`	`0110011`

TABLE 10 – Codage de `add x9, x20, x21`

4.3 Adapter et implémenter les instructions RISC-V (§ 4.1.2 du sujet)

4.3.1 Modes d'adressage

Le mode d'adressage indique où se trouve l'opérande d'une instruction (tableau 11).

Mode	Principe	Exemples
Immédiat	L'opérande est une constante codée dans l'instruction elle-même.	`addi x1, x0, -7`
Direct	L'adresse mémoire de l'opérande est fixée dans l'instruction ; ici, la base `x0` vaut 0.	`lw x1, 64(x0)`
Indirect	Un registre contient l'adresse de l'opérande en mémoire (pointeur).	`lw x1, 0(x2)` `sw x3, 4(x2)`

TABLE 11 – Modes d'adressage

En RISC-V, `lw` et `sw` n'ont qu'un seul mode : l'adresse est la somme d'un registre de base `rs1` et d'un déplacement signé de 12 bits, ce qui s'écrit `offset(rs1)`. L'adressage direct en est le cas particulier `rs1` = `x0` : `x0` vaut toujours 0, donc l'adresse 64 de `lw x1, 64(x0)` est entièrement codée dans l'instruction. L'adressage indirect en est le cas `offset` = 0. Dans ce modèle, la mémoire est adressée par mots de 32 bits (`memory[addr]`) et non par octets comme dans la spécification : `4(x2)` désigne le mot situé 4 mots après l'adresse contenue dans `x2`.

4.3.2 Corrections du modèle fourni

Pour tester les immédiats négatifs, les accès mémoire avec un déplacement positif et négatif, et le registre `x0`, nous avons écrit le programme `test_base` (code 7). Ses résultats sont donnés dans le tableau 12, relevés dans ModelSim avec la commande `examine` (figure 10) : le modèle d'origine présente trois erreurs.

Code 7 – Programme de test `test_base.s`

```
addi x1, x0, -7       # x1 = -7
addi x2, x0, 100      # x2 = 100
sub  x3, x2, x1       # x3 = 107
andi x4, x1, -16      # x4 = -16
ori  x5, x0, -1       # x5 = -1
addi x6, x0, 3        # x6 = 3
sll  x7, x2, x6       # x7 = 800
srl  x8, x5, x6       # x8 = 0x1FFFFFFF = 536870911
sw   x3, 4(x2)        # mot 104 = 107
sw   x1, -4(x2)       # mot 96 = -7
lw   x9, 4(x2)        # x9 = 107
lw   x10, -4(x2)      # x10 = -7
addi x0, x0, 5        # x0 doit rester a 0
add  x11, x0, x0      # x11 = 0
ebreak
```

	Instruction	Attendu	Modèle d'origine	Après correction
x0	`addi x0, x0, 5`	0	5	0
x1	`addi x1, x0, -7`	−7	4089	−7
x3	`sub x3, x2, x1`	107	−3989	107
x4	`andi x4, x1, -16`	−16	4080	−16
x5	`ori x5, x0, -1`	−1	4095	−1
x8	`srl x8, x5, x6`	536870911	511	536870911
x9	`lw x9, 4(x2)`	107	4089	107
x10	`lw x10, -4(x2)`	−7	4089	−7
x11	`add x11, x0, x0`	0	10	0
mot 104	`sw x3, 4(x2)`	107	X	107
mot 96	`sw x1, -4(x2)`	−7	X	−7
mot 0	première instruction	`ff900093`	4089	`ff900093`

TABLE 12 – Résultats de `test_base` (x2 = 100, x6 = 3 et x7 = 800 sont corrects dans les deux cas)

```
# === AVANT correction (modele d'origine) ===
VSIM 50> examine -radix decimal /system/regs
# {5 4089 100 -3989 4080 4095 3 800 511 4089 4089 10 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
VSIM 51> examine -radix decimal {/system/memory[0]} {/system/memory[96]} {/system/memory[104]}
# 4089 x x
VSIM 52> quit -sim
ModelSim> vlog -sv riscvproc-2.sv
# End time: 20:31:58 on Oct 26,2026, Elapsed time: 0:00:09
# Errors: 0, Warnings: 0
# Model Technology ModelSim SE-64 vlog 2020.4 Compiler 2020.10 Oct 13 2020
# Start time: 20:31:58 on Oct 26,2026
# vlog -reportprogress 300 -sv riscvproc-2.sv
# -- Compiling module system
#
# Top level modules:
#       system
# End time: 20:31:58 on Oct 26,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
ModelSim> vsim -voptargs=+acc -onfinish stop work.system
# vsim -voptargs="+acc" -onfinish stop work.system
# Start time: 20:31:59 on Oct 26,2026
# ** Note: (vsim-3813) Design is being optimized due to module recompilation...
# Loading sv_std.std
# Loading work.system(fast)
VSIM 55> run -all
# erreur: mauvaise valeur dans op-code
# ** Note: $finish    : riscvproc-2.sv(149)
#    Time: 160 ns  Iteration: 0  Instance: /system
# Break in Module system at riscvproc-2.sv line 149
VSIM 56> echo "=== APRES correction ==="
# === APRES correction ===
VSIM 57> examine -radix decimal /system/regs
# {0 -7 100 107 -16 -1 3 800 536870911 107 -7 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
VSIM 58> examine -radix hexadecimal {/system/memory[0]}
# ff900093
VSIM 59> examine -radix decimal {/system/memory[96]} {/system/memory[104]}
# -7 107

VSIM 60>
```

FIGURE 10 – Transcript ModelSim : `test_base` avec le modèle d'origine, puis avec le modèle corrigé

Le message « erreur : mauvaise valeur dans op-code » affiché au lancement ne vient pas du programme : ce modèle lit l'instruction par une affectation non bloquante, et l'instruction n'est pas encore définie à la première période (section 4.4).

Immédiats non étendus en signe. Le modèle définissait `` `define I_IMM $signed(instr[31:20]) ``. Or, en Verilog, dès qu'un opérande d'une expression est non signé (ici le registre `regs[rs1]`), toute l'expression est évaluée en non signé et l'immédiat de 12 bits est complété par des zéros : -7 (`0xFF9`) devient 4089. C'est l'erreur sur x1, x3, x4 et x5. Nous faisons l'extension de signe explicitement, en recopiant le bit 31 par concaténation, pour le format I comme pour le format S (code 8).

Adresse des accès mémoire. L'adresse était calculée par `addr = `LOAD_OFFSET + regs[`RS1];` avec `` `define LOAD_OFFSET instr[31:20] << 12 ``. Une macro est une simple substitution de texte : la ligne devient `addr = instr[31:20] << 12 + regs[`RS1];`, et comme + est prioritaire sur <<, le déplacement est décalé de 12 + `rs1` bits. Dans nos programmes, l'adresse de 12 bits valait donc toujours 0 : les `lw` lisaient le mot 0 et les `sw` écrasaient la première instruction du programme (d'où x9 = x10 = 4089 et le mot 0 égal à 4089). Le décalage de 12 bits est de toute façon faux : il correspond au format de `lui`, pas aux accès mémoire. L'adresse vaut maintenant `rs1` + immédiat I pour `lw`, et `rs1` + immédiat S pour `sw`.

Registre x0. `addi x0, x0, 5` modifiait `x0`, qui doit toujours valoir 0 (spécification, chapitre 2.1). Nous remettons `regs[0]` à 0 à la fin de chaque instruction.

Code 8 – Corrections du modèle (immédiats, adresses, x0)

```
`define I_IMM   {{20{instr[31]}}, instr[31:20]}              // I-type immediate
`define S_IMM   {{20{instr[31]}}, instr[31:25], instr[11:7]}  // S-type immediate
...
`LW: begin
  addr      = regs[`RS1] + `I_IMM;
  regs[`RD] = memory[addr];
end
...
`SW: begin
  addr         = regs[`RS1] + `S_IMM;
  memory[addr] = regs[`RS2];
end
...
regs[0] = 0;   // x0 is hardwired to 0
```

4.3.3 Ajout des instructions de l'extension M

L'extension M ajoute huit instructions de multiplication et de division entières (tableau 13). Ce sont toutes des instructions de format R, avec le même opcode `0110011` que `add` ; elles se distinguent des opérations de RV32I par `funct7` = `0000001` (au lieu de `0000000` ou `0100000`), et `funct3` choisit l'opération.

Instruction	funct3	Résultat
`MUL`	`000`	32 bits de poids faible de `rs1` × `rs2`
`MULH`	`001`	32 bits de poids fort, signé × signé
`MULHSU`	`010`	32 bits de poids fort, `rs1` signé × `rs2` non signé
`MULHU`	`011`	32 bits de poids fort, non signé × non signé
`DIV`	`100`	quotient signé, arrondi vers 0
`DIVU`	`101`	quotient non signé
`REM`	`110`	reste signé, du signe du dividende
`REMU`	`111`	reste non signé

TABLE 13 – Instructions de l'extension M (opcode `0110011`, funct7 = `0000001`)

Les modifications à apporter au processeur sont donc les suivantes :

— **Décodage** : aucun nouvel opcode n'est nécessaire. Le processeur décode déjà les opérations entre registres sur le couple {`funct3`, `funct7`} ; il suffit d'y ajouter les huit combinaisons.

— **Chemin de données** : un multiplieur 32×32 bits produisant 64 bits, et un diviseur donnant le quotient et le reste, en versions signée et non signée. Les 32 bits de poids faible du produit ne dépendent pas du signe des opérandes, d'où une seule instruction `MUL` ; les 32 bits de poids fort en dépendent, d'où les trois variantes `MULH`.

— **Cas particuliers** : RISC-V ne déclenche pas d'exception lors d'une division ; la spécification fixe le résultat de la division par zéro et du dépassement (tableau 14).

Cas	`DIV`	`DIVU`	`REM`	`REMU`
Division par 0	-1	$2^{32}-1$	dividende	dividende
Dépassement $-2^{31}/(-1)$	-2^{31}	–	0	–

TABLE 14 – Résultats imposés par la spécification pour la division

4.3.4 Implémentation et validation de l'extension M

Nous avons ajouté les huit codes sous la même forme que les instructions existantes, par exemple `` `define MUL {3'b000, 7'b0000001} ``, et huit cas dans le décodage des opérations entre registres (code 9). Trois points de Verilog demandent de l'attention :

— **Largeur et extension des opérandes.** Dans une affectation directe telle que `prod = a * b`, une destination de 64 bits participe au calcul de la largeur de l'expression : deux opérandes de 32 bits ne provoquent donc pas à eux seuls une troncature à 32 bits [3]. Nos concaténations rendent surtout explicite l'extension voulue pour chaque opérande : copie du signe avec `{{32{x[31]}}, x}` ou ajout de zéros avec `{32'b0, x}`. Le produit est conservé sur 64 bits dans `prod`, puis les bits 63 à 32 sont copiés pour les variantes `MULH`.

— **Division signée.** Les deux opérandes passent par `$signed`. La division signée de Verilog arrondit vers zéro et le reste prend le signe du dividende, comme en RISC-V.

— **Cas particuliers.** En Verilog, une division par zéro donne `X` ; ces cas sont donc traités avant la division, avec `if`/`else`. Nous avons évité l'opérateur conditionnel `? :` : si l'une de ses branches est non signée (par exemple `32'hFFFFFFFF`), toute l'expression devient non signée, et la division de l'autre branche est alors calculée en non signé ($-7/2$ donne 2147483644 au lieu de -3).

Code 9 – Extrait de l'implémentation de l'extension M

```
reg[63:0] prod;    // 64-bit product for MULH, MULHSU, MULHU
...
`MUL:     regs[`RD] = regs[`RS1] * regs[`RS2];  // 32-bit target: low half
`MULH:    begin
            prod = {{32{regs[`RS1][31]}}, regs[`RS1]} * {{32{regs[`RS2][31]}}, regs[
    `RS2]};
            regs[`RD] = prod[63:32];
          end
...
`DIV:     if (regs[`RS2] == 0)
            regs[`RD] = 32'hFFFFFFFF;                       // division by 0: -1
          else if (regs[`RS1] == 32'h80000000 && regs[`RS2] == 32'hFFFFFFFF)
            regs[`RD] = 32'h80000000;                       // overflow: -2^31
          else
            regs[`RD] = $signed(regs[`RS1]) / $signed(regs[`RS2]);
```

Le programme `test_rv32im` teste les huit instructions et les trois cas particuliers. Avant la modification, le processeur affichait onze fois « erreur : mauvaise instruction arithmétique », une fois par instruction de l'extension M, et les registres correspondants restaient à 0 (figure 11). Après la modification, toutes les valeurs sont celles attendues (tableau 15 et figure 12).

```
VSIM 33> run -all
# erreur: mauvaise valeur dans op-code
# erreur: mauvaise instruction arithmÃ©tique
# erreur: mauvaise instruction arithmÃ©tique
# erreur: mauvaise instruction arithmÃ©tique
# erreur: mauvaise instruction arithmÃ©tique
# erreur: mauvaise instruction arithmÃ©tique
# erreur: mauvaise instruction arithmÃ©tique
# erreur: mauvaise instruction arithmÃ©tique
# erreur: mauvaise instruction arithmÃ©tique
# erreur: mauvaise instruction arithmÃ©tique
# erreur: mauvaise instruction arithmÃ©tique
# erreur: mauvaise instruction arithmÃ©tique
# ** Note: $finish    : riscvproc-2.sv(153)
#    Time: 190 ns  Iteration: 0  Instance: /system
# Break in Module system at riscvproc-2.sv line 153
VSIM 34> examine -radix decimal /system/regs
# {0 -7 2 0 0 0 0 0 0 0 0 -2147483648 -1 31 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}

VSIM 35>
```

FIGURE 11 – Transcript ModelSim : `test_rv32im` avant l'ajout de l'extension M (le transcript affiche mal l'accent de « arithmétique »)

Instruction	Registre	Résultat	Remarque
`mul x3, x1, x2`	x3	-14	
`div x4, x1, x2`	x4	-3	arrondi vers 0
`rem x5, x1, x2`	x5	-1	signe du dividende
`divu x6, x1, x2`	x6	2147483644	`0xFFFFFFF9` / 2
`div x7, x1, x0`	x7	-1	division par 0
`mulh x8, x1, x2`	x8	-1	partie haute de -14
`mulhu x9, x1, x2`	x9	1	`0xFFFFFFF9` $\times$ 2 = `0x1FFFFFFF2`
`remu x10, x1, x2`	x10	1	
`mulhsu x16, x1, x2`	x16	-1	
`div x14, x11, x12`	x14	-2^{31}	dépassement
`rem x15, x11, x12`	x15	0	dépassement

TABLE 15 – Résultats de `test_rv32im`, avec x1 = -7, x2 = 2, x11 = -2^{31} et x12 = -1

```
VSIM 66> examine -radix decimal /system/regs
# {0 -7 2 -14 -3 -1 2147483644 -1 -1 1 1 -2147483648 -1 31 -2147483648 0 -1 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
```

FIGURE 12 – Transcript ModelSim : registres après `test_rv32im`

4.4 Adapter la micro-architecture du processeur RISC-V (§ 4.2 du sujet)

Pour pouvoir relier le processeur au bloc `quadram`, le module `system`, qui n'avait pas de port, devient le module `riscvproc` avec une horloge, un reset et un bus de données (tableau 16). L'adresse fait 12 bits, comme le bus d'adresse interne du processeur : elle désigne 4096 mots, soit les 4 RAM de 1024 mots de la partie 5.

Port	Direction	Largeur	Rôle
`clock`	entrée	1	horloge
`reset`	entrée	1	remise à zéro synchrone, active à 1
`address`	sortie	12	adresse du mot de données
`dataToWrite`	sortie	32	donnée à écrire
`dataToRead`	entrée	32	donnée lue
`we`	sortie	1	autorisation d'écriture

TABLE 16 – Ports ajoutés au processeur

Les autres modifications sont les suivantes (code 10) :

— **Horloge.** Le paramètre `Step` et l'attente `#Step` sont remplacés par `@(negedge clock)` : le processeur exécute une instruction par période d'horloge, sur le front descendant. Le bloc `quadram` échantillonne ses entrées sur le front montant, une demi-période plus tard : les signaux du bus sont alors stables.

— **Reset synchrone**, actif à 1 : sur un front descendant avec `reset` = 1, `pc` et les registres reprennent leur valeur initiale, 0. La mémoire programme n'est pas modifiée.

— **Lecture de l'instruction par affectation bloquante.** Le modèle d'origine utilisait `instr <= memory[pc];`. Avec une affectation non bloquante, `instr` n'est mise à jour qu'après le décodage, si bien que chaque période décodait l'instruction lue à la période précédente ; à la première période, l'instruction était encore indéfinie, d'où un message « mauvaise valeur dans op-code » au démarrage. Avec `instr = memory[pc];`, l'instruction est lue et exécutée dans la même période. C'est aussi nécessaire pour la partie 5, où `lw` et `sw` attendent la mémoire alors que `instr` doit rester stable.

— **Sorties du bus** initialisées à 0 : à cette étape, les données sont encore dans la mémoire interne.

Code 10 – Boucle principale du processeur avec horloge et reset

```
always begin // one instruction per clock period
  @(negedge clock);
  if (reset) begin
    // synchronous reset: same values as at the start of the simulation
    pc = 0;
    for (index = 0; index < NRegs; index = index + 1) begin
      regs[index] = 0;
    end
  end else begin
    instr = memory[pc];    // instruction fetch (blocking: executed in this same
   period)
    pc    = pc + 1;        // increment program counter
    addr  = 0;             // address initial value
    case(`OPCODE)          // decode instruction
      ...
    endcase
    regs[0] = 0;
  end // if (reset) ... else
end
```

Le testbench `tb_riscvproc.sv` instancie le processeur, génère une horloge de période 10 ns et maintient le reset à 1 jusqu'à 12 ns, ce qui laisse un front descendant de reset à 10 ns. La première instruction est exécutée à 20 ns, puis une instruction toutes les 10 ns. Sur la figure 13, les curseurs marquent la fin du reset (12 ns), la première instruction (20 ns) et la fin de la simulation (190 ns). Les 170 ns qui séparent la première et la dernière des 18 instructions de `test_rv32im` représentent 17 périodes : une instruction par période. Les résultats sont identiques à ceux de la section précédente : `test_rv32im` se termine à 190 ns, `test_base` à 160 ns, `instr.txt` et `instr2.txt` à 70 ns (figure 14).

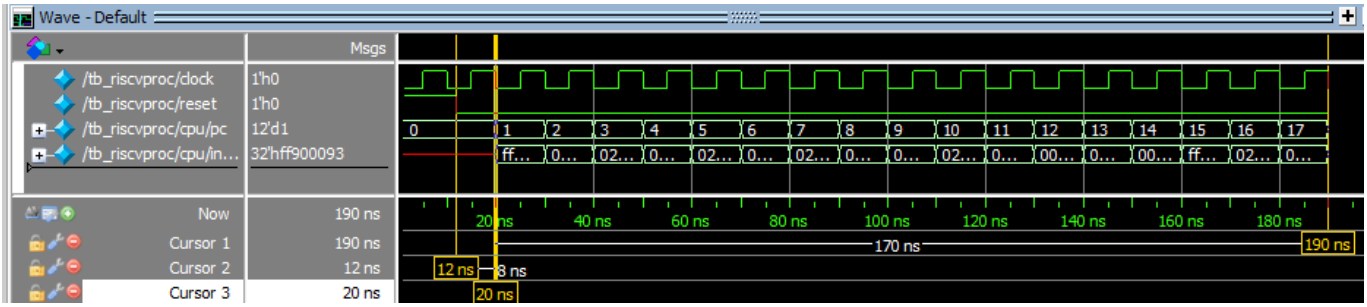


FIGURE 13 – Processeur avec horloge et reset, programme `test_rv32im` : une instruction par période

```
VSIM 40> run -all
# ** Note: $finish    : riscvproc.sv(219)
#    Time: 160 ns  Iteration: 1  Instance: /tb_riscvproc/cpu
# Break in Module riscvproc at riscvproc.sv line 219
VSIM 41> examine -radix decimal /tb_riscvproc/cpu/regs
# {0 -7 100 107 -16 -1 3 800 536870911 107 -7 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
VSIM 42> quit -sim
ModelSim> vsim -quiet -voptargs=+acc -onfinish stop work.tb_riscvproc +prog=../../programmes/instr.txt
# End time: 18:02:50 on Oct 27,2026, Elapsed time: 0:00:10
# Errors: 0, Warnings: 0
# vsim -quiet -voptargs="+acc" -onfinish stop work.tb_riscvproc "+prog=../../programmes/instr.txt"
# Start time: 18:02:50 on Oct 27,2026
# ** Note: (vsim-8009) Loading existing optimized design _opt
VSIM 44> run -all
# ** Note: $finish    : riscvproc.sv(219)
#    Time: 70 ns  Iteration: 1  Instance: /tb_riscvproc/cpu
# Break in Module riscvproc at riscvproc.sv line 219
VSIM 45> examine -radix decimal /tb_riscvproc/cpu/regs
# {0 1 6 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
VSIM 46> quit -sim
ModelSim> vsim -quiet -voptargs=+acc -onfinish stop work.tb_riscvproc +prog=instr2.txt
# End time: 18:03:00 on Oct 27,2026, Elapsed time: 0:00:10
# Errors: 0, Warnings: 0
# vsim -quiet -voptargs="+acc" -onfinish stop work.tb_riscvproc "+prog=instr2.txt"
# Start time: 18:03:00 on Oct 27,2026
# ** Note: (vsim-8009) Loading existing optimized design _opt
VSIM 48> run -all
# ** Note: $finish    : riscvproc.sv(219)
#    Time: 70 ns  Iteration: 1  Instance: /tb_riscvproc/cpu
# Break in Module riscvproc at riscvproc.sv line 219
VSIM 49> examine -radix decimal /tb_riscvproc/cpu/regs
# {0 1 6 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}

VSIM 50>
```

FIGURE 14 – Transcript ModelSim : processeur avec horloge, programmes `test_base`, `instr.txt` puis `instr2.txt`

5 Création du système

Dans le système final (figure 15), le programme reste dans la mémoire interne du processeur et les données sont stockées dans le bloc `quadram` de la partie 3. Le bloc de plus haut niveau, `systeme`, est écrit en VHDL : il instancie le processeur Verilog en tant que « component » ainsi que le bloc `quadram`, et les relie par des signaux internes.

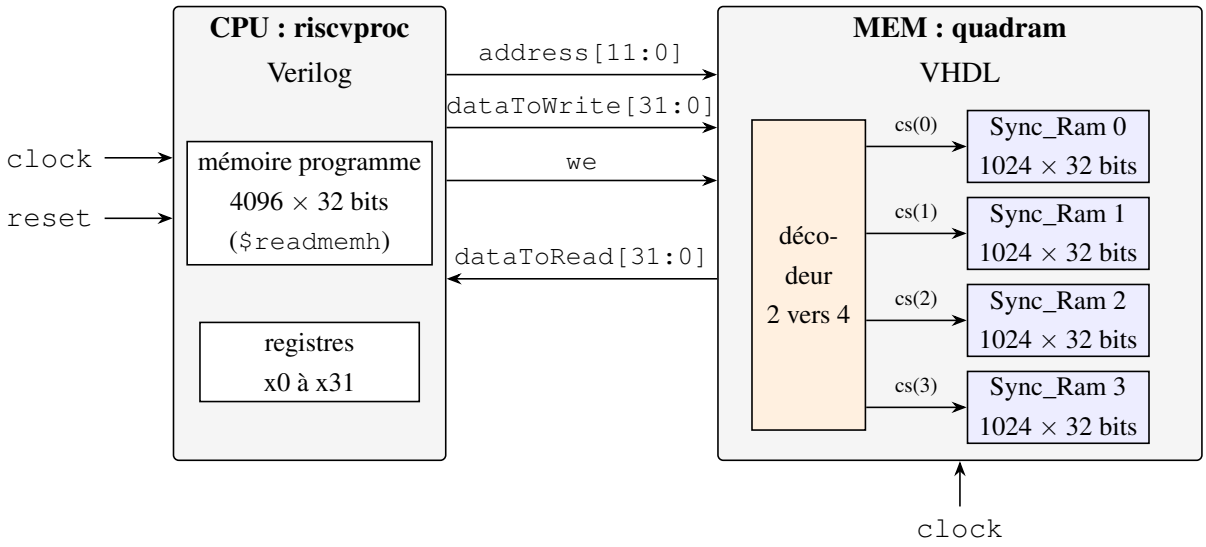


FIGURE 15 – Système final : processeur et bloc `quadram`. Les bits 11–10 de l'adresse attaquent le décodeur ; les bits 9–0, la donnée à écrire, `we` et l'horloge sont communs aux quatre RAM, dont les sorties sont reliées (bus trois états).

5.1 Accès aux données dans la mémoire externe

Les données n'étant plus dans `memory`, seules les instructions de manipulation des données changent : `lw` et `sw` passent maintenant par le bus (code 11). `sw` place l'adresse, la donnée et `we` = 1 sur le bus ; `lw` place l'adresse, puis recopie `dataToRead` dans le registre destination. Les autres instructions ne sont pas modifiées.

Code 11 – Instructions `lw` et `sw` reliées au bloc `quadram`

```
`LW: begin
  address    = regs[`RS1] + `I_IMM;    // address on the bus (falling edge)
  @(negedge clock);                    // rising edge in between: the quadram samples
    the address
  regs[`RD] = dataToRead;              // the word read is valid now
end
...
`SW: begin
  address     = regs[`RS1] + `S_IMM; // address and data on the bus
  dataToWrite = regs[`RS2];
  we          = 1;
  @(negedge clock);                    // rising edge in between: the quadram writes
    the word
  we          = 0;
end
```

5.2 Synchronisation avec le bloc quadram

Le bloc `quadram` est synchrone : l'écriture a lieu sur le front montant de l'horloge, et la lecture passe par le registre d'adresse `read_address` de `Sync_Ram` : la donnée lue ne correspond à une nouvelle adresse qu'après le front montant qui a mémorisé cette adresse. Le processeur travaillant sur le front descendant, un accès se déroule ainsi :

1. front descendant : le processeur place l'adresse sur le bus et, pour `sw`, la donnée et `we` = 1 ;
2. front montant suivant : la RAM sélectionnée écrit la donnée, ou mémorise l'adresse de lecture ;
3. front descendant suivant : `lw` recopie `dataToRead`, valide depuis le front montant ; `sw` remet `we` à 0.

L'attente est réalisée par un second `@(negedge clock)` dans l'instruction : `lw` et `sw` durent deux périodes, les autres instructions une seule. Le tableau 17 et la figure 16 montrent cette séquence pour `test_base` exécuté sur le système. À 140 ns (premier curseur), `lw x9, 4(x2)` place l'adresse 104, mais `dataToRead` vaut encore -7, le mot 96 écrit juste avant ; la donnée 107 n'apparaît qu'après le front montant de 145 ns (second curseur). Sans l'attente, `lw` lirait donc l'ancien mot.

Temps	Front	Instruction	Bus et registres
100 ns	↓	`sw x3, 4(x2)`	`address` = 104, `dataToWrite` = 107, `we` = 1
105 ns	↑		la RAM 0 écrit 107 au mot 104
110 ns	↓		`we` = 0
120 ns	↓	`sw x1, -4(x2)`	`address` = 96, `dataToWrite` = −7, `we` = 1
140 ns	↓	`lw x9, 4(x2)`	`address` = 104, `dataToRead` = −7 (mot 96)
145 ns	↑		adresse 104 mémorisée : `dataToRead` = 107
150 ns	↓		x9 = 107
160 ns	↓	`lw x10, -4(x2)`	instruction suivante

TABLE 17 – Accès à la mémoire pendant `test_base` sur le système

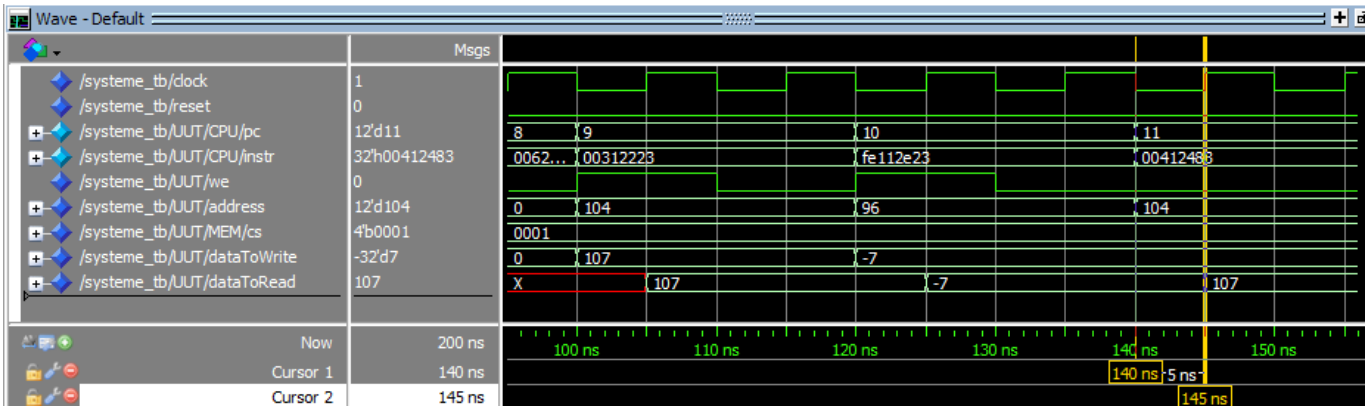


FIGURE 16 – Écritures puis lecture dans le bloc `quadram` : la donnée lue n'est valide qu'après le front montant (curseurs à 140 et 145 ns)

5.3 Micro-architecture du processeur

Les ports, l'horloge et le reset nécessaires à cet interfaçage ont été ajoutés à la section 4.4. Comme le processeur travaille sur le front descendant et la mémoire sur le front montant, aucune autre modification n'a été nécessaire.

5.4 Bloc système en VHDL

Le fichier `systeme.vhd` (code 12) contient :

— la déclaration du `component riscvproc`, avec le même nom et les mêmes ports que le module Verilog, écrits avec des types VHDL (`std_logic` et `std_logic_vector`) ;

— les signaux internes qui forment le bus entre les deux blocs ;

— l'instance du bloc `quadram` générique de la partie 3, avec `Nbits_sel` = 2, `Nbits_addr` = 10 et `Nbits_data` = 32 : quatre RAM de 1024 mots de 32 bits, soit 4096 mots adressés sur 12 bits, comme le bus d'adresse du processeur ;

— les conversions de type : les ports de `quadram` sont de type `unsigned`, ceux du processeur de type `std_logic_vector`.

Code 12 – Extrait de `systeme.vhd`

```
component riscvproc is
  port (
    clock       : in  std_logic;
    reset       : in  std_logic;
    address     : out std_logic_vector(11 downto 0);
    dataToWrite : out std_logic_vector(31 downto 0);
    dataToRead  : in  std_logic_vector(31 downto 0);
    we          : out std_logic
  );
end component riscvproc;
...
CPU: riscvproc
  port map (clock => clock, reset => reset, address => address,
            dataToWrite => dataToWrite, dataToRead => dataToRead, we => we);

-- 4 RAMs of 1024 words of 32 bits: address(11 downto 10) selects the RAM,
-- address(9 downto 0) is the word inside the RAM
MEM: entity work.quadram(RTL)
  generic map (Nbits_sel => 2, Nbits_addr => 10, Nbits_data => 32)
  port map (clock => clock, we => we,
            address => unsigned(address),
            datain  => unsigned(dataToWrite),
            dataout => dataOut);

-- the quadram works with unsigned, the processor with std_logic_vector
dataToRead <= std_logic_vector(dataOut);
```

Le testbench `systeme_tb.vhd` génère l'horloge (période 10 ns) et le reset (à 1 jusqu'à 12 ns) ; un processus de surveillance arrête la simulation au bout de 100 μs si le programme n'atteint jamais `ebreak`. Les blocs des parties 2 et 3 sont réutilisés sans copie : ils sont compilés depuis leurs dossiers, dans la même bibliothèque `work` que le processeur (code 13).

Code 13 – Compilation et simulation du système dans ModelSim

```
vlib work
vlog -sv riscvproc.sv
vcom -93 ../p2_q5/ramchip.vhd ../p3_q4/decoder.vhd ../p3_q4/quadram_for.vhd systeme.
    vhd systeme_tb.vhd
vsim -voptargs=+acc -onfinish stop work.systeme_tb
run -all
```

ModelSim associe le `component` au module Verilog de même nom présent dans la bibliothèque. Si ce module n'a pas été compilé, ModelSim ne donne qu'un avertissement (« Component instance "CPU : riscvproc" is not bound ») et simule le système sans processeur : c'est ce qui nous est arrivé au premier essai, et la simulation s'est alors arrêtée sur le délai de surveillance de 100 μs.

5.5 Validation

Programme instr2. À la fin du programme, x1 = 1 et x2 = 6, et le mot 65 se trouve maintenant dans la RAM 0 du bloc `quadram` (`RAMS(0)/RAM/ram(65)` = 6). Sur la figure 17, `sw x2, 64(x1)` place à 60 ns l'adresse 65, la donnée 6 et `we` = 1 ; après le front montant de 65 ns, `dataToRead` relit 6 à cette adresse. La simulation se termine à 80 ns au lieu de 70 ns, car l'instruction `sw` dure deux périodes.

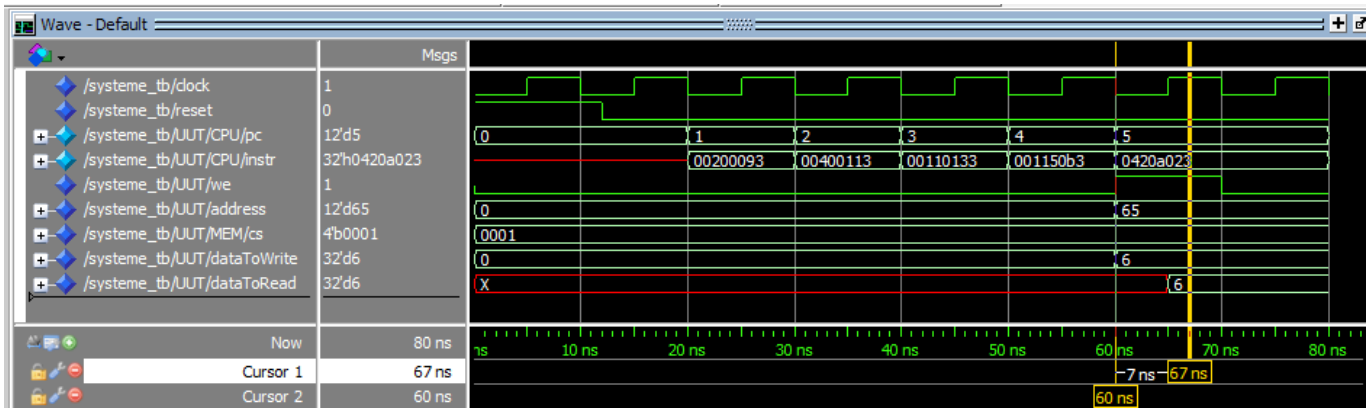


FIGURE 17 – Système complet avec le programme `instr2`

Accès aux quatre RAM. Le programme `test_memoire` écrit 11, 22, 33 et 44 au même déplacement 5 dans chacune des quatre RAM (mots 5, 1029, 2053 et 3077), puis relit ces mots ; il teste aussi un déplacement négatif, la frontière entre les RAM 0 et 1 (mot 1023) et une lecture juste après une écriture à la même adresse. Les résultats sont ceux attendus (tableau 18). Pendant les quatre écritures, le signal `cs` du décodeur prend successivement les valeurs `0001`, `0010`, `0100` et `1000` (figure 18) ; les curseurs, à 90, 110, 130 et 150 ns, montrent que chaque `sw` dure deux périodes. Avant chaque écriture, `dataToRead` vaut X, car la RAM sélectionnée ne contient encore rien au mot lu ; la valeur écrite apparaît au front montant suivant. La simulation se termine à 350 ns. Le programme compte 23 instructions, dont 11 accès mémoire qui ajoutent chacun une période. Entre la première instruction et `ebreak`, il y a donc $23 - 1 + 11 = 33$ périodes. Comme la première instruction commence à 20 ns et que `ebreak` arrête la simulation dès son exécution, on obtient :

$$t_{\text{fin}} = 20 + (23 - 1 + 11) \times 10 = 350 \text{ ns}.$$

Instructions	Destination	Résultat
4 `sw` au déplacement 5 de chaque RAM	mots 5, 1029, 2053, 3077	11, 22, 33, 44
4 `lw` de ces mots	x20, x21, x22, x23	11, 22, 33, 44
`add` des quatre valeurs	x24	110
`lw x25, -5(x8)`, avec x8 = 3082	x25	44
`sw x24, -1(x5)`, avec x5 = 1024	mot 1023	110
`lw x26, 1023(x0)`	x26	110

TABLE 18 – Résultats de `test_memoire` sur le système

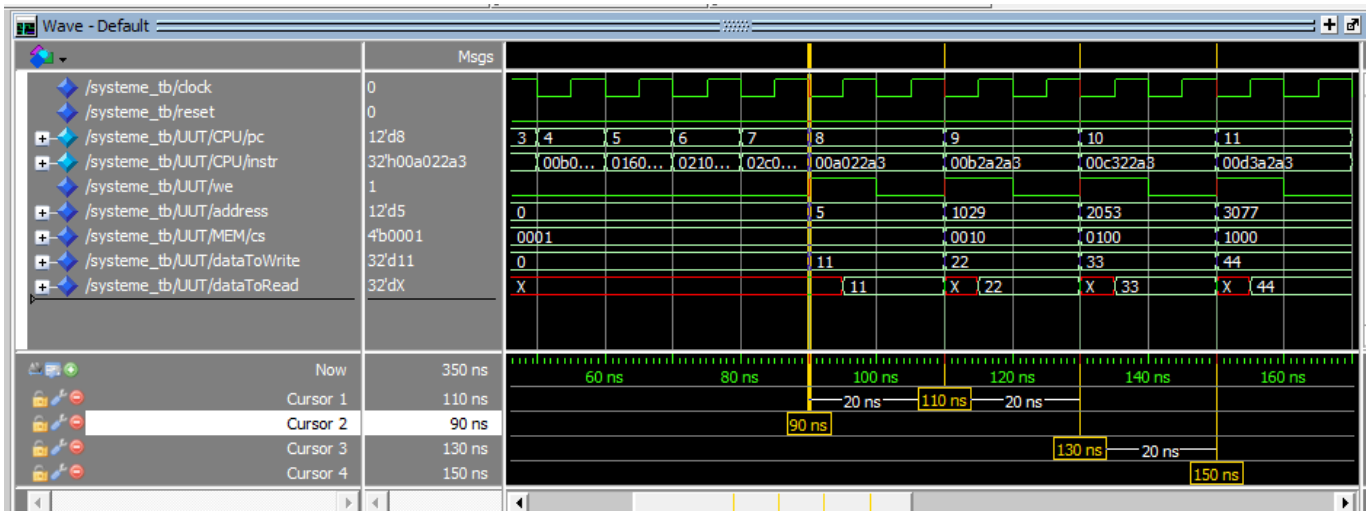


FIGURE 18 – Les quatre écritures de `test_memoire` : le décodeur sélectionne successivement les quatre RAM

Lors des premiers essais d'intégration, ce test a révélé un défaut de la version de `Sync_Ram` alors utilisée : l'écriture n'était pas conditionnée par `cs` (`if we = '1' then`). Comme les quatre RAM partagent l'adresse, la donnée et `we`, chaque `sw` écrivait dans les quatre RAM au même déplacement, et les quatre mots relus valaient tous 44 (x24 = 176 au lieu de 110). La condition est devenue `if we = '1' and cs = '1' then`. Les essais de la RAM avec sélection et de `quadram` présentés dans les parties 2 et 3 utilisent la version corrigée. Le testbench de `quadram` détecte aussi cette erreur (messages d'erreur sur les RAM 0 à 2) lorsqu'il est exécuté avec la RAM non corrigée.

Programme test_base. Les registres ont les mêmes valeurs qu'à la section 4.4, et les mots 104 et 96 se trouvent maintenant dans la RAM 0 du bloc `quadram` (107 et −7). La simulation se termine à 200 ns au lieu de 160 ns : les quatre accès à la mémoire durent chacun une période de plus.

5.6 Différences avec le premier programme

Le premier programme, `instr.txt` (section 4.1), charge ses deux opérandes avec deux `lw` depuis les mots 64 et 65, où `$readmemh` a placé les données 2 et 4. Dans `instr2.txt`, deux `addi` fournissent directement les mêmes valeurs ; les quatre instructions suivantes restent identiques (tableau 19).

Mot	`instr.txt`	`instr2.txt`	Effet attendu
0	`04002083` `lw x1, 64(x0)`	`00200093` `addi x1, x0, 2`	x1 = 2
1	`04102103` `lw x2, 65(x0)`	`00400113` `addi x2, x0, 4`	x2 = 4
2	`00110133 : add x2, x2, x1`		x2 = 6
3	`001150b3 : srl x1, x2, x1`		x1 = 1
4	`0420a023 : sw x2, 64(x1)`		mot 65 = 6
5	`00100073 : ebreak`		arrêt

TABLE 19 – Comparaison des deux programmes : seules les deux premières instructions changent

Sur le processeur corrigé, les deux programmes donnent x1 = 1, x2 = 6 et `memory[65]` = 6. Pour `instr.txt`, `$readmemh` initialise aussi les données : les deux `lw` retrouvent donc 2 et 4 dans la mémoire interne.

Dans le système, `$readmemh` charge toujours cette mémoire interne, alors que `lw` lit dans `quadram`. Les valeurs placées à `@40` n'y sont pas transférées. Avec `instr.txt`, x1 et x2 sont indéfinis et le résultat est incorrect, bien que `ebreak` soit atteint. Avec `instr2.txt`, les deux `addi` fournissent les constantes et `sw` écrit le résultat dans la RAM externe.

Modèle	Programme	x1	x2	Fin
Processeur corrigé, avec horloge	`instr.txt`	1	6	70 ns
Processeur corrigé, avec horloge	`instr2.txt`	1	6	70 ns
Système, RAM externes non initialisées	`instr.txt`	indéfini	indéfini	100 ns
Système	`instr2.txt`	1	6	80 ns

TABLE 20 – Exécution des deux programmes avec mémoire interne puis externe

Les deux `lw` du premier programme ajoutent deux périodes, soit 20 ns par rapport à `instr2.txt`. Pour conserver `instr.txt`, il faudrait initialiser explicitement les mots 64 et 65 de `quadram` ; nous avons retenu le second programme pour valider le système sans ce préchargement.

Conclusion

Le système obtenu associe un processeur Verilog à quatre RAM VHDL de 1024 mots de 32 bits. Le même modèle de RAM est utilisé seul, dans les tests de `quadram` et dans le système final. Les paramètres permettent d'en changer les dimensions sans réécrire son architecture.

Les essais ont surtout mis en évidence deux conditions d'intégration : seule la RAM sélectionnée doit accepter une écriture, et le processeur doit attendre la mémorisation de l'adresse avant de lire une donnée. Le programme `test_memoire` vérifie la sélection des quatre blocs, un déplacement négatif et une frontière entre deux RAM. La comparaison de `instr.txt` et `instr2.txt` montre aussi que déplacer les données vers une mémoire externe oblige à revoir leur initialisation.

Les tests de validation du processeur corrigé et du système donnent les résultats attendus. La vérification finale reproduit également les défauts du modèle d'origine et l'échec du premier programme lorsque les données externes ne sont pas initialisées. Cette validation reste celle des cas testés : le processeur implémente un sous-ensemble de RV32I et les huit instructions de l'extension M, et non toute l'ISA RV32IM.

Pour aller jusqu'à une implantation sur FPGA, il faudrait remplacer les attentes comportementales par une machine à états synchrone, utiliser une sélection de données adaptée aux ressources internes de la cible, puis vérifier la synthèse, les contraintes d'horloge et le fonctionnement sur carte. Pour exécuter du code issu d'un compilateur C RISC-V, il faudrait compléter les instructions, passer à l'adressage par octets et prévoir le démarrage du programme, la pile et la répartition des mémoires. Ces deux prolongements n'ont pas été réalisés.

Table des figures

1 RAM initiale : écritures de 4 et 7, puis lectures aux adresses 1 et 2 5
2 RAM avec ports unsigned : même chronologie que le modèle initial 5
3 Configuration 1024 mots de 32 bits : même testbench et résultat TRUE 7
4 CS asynchrone : sortie à haute impédance et écriture bloquée entre 67 et 87 ns 7
5 Décodeur 2 vers 4 : une sélection par adresse valide et aucune sélection pour XX 8
6 Quadram fixe : écritures puis lectures indépendantes des quatre RAM 10
7 Version générique : huit RAM de 16 mots de 16 bits, test des huit sélections 11
8 Configuration du système final : quatre RAM de 1024 mots de 32 bits 11
9 Transcript ModelSim : `instr.txt` avec le modèle d'origine, puis avec le modèle corrigé 14
10 Transcript ModelSim : `test_base` avec le modèle d'origine, puis avec le modèle corrigé 16
11 Transcript ModelSim : `test_rv32im` avant l'ajout de l'extension M (le transcript affiche mal l'accent de « arithmétique ») 19
12 Transcript ModelSim : registres après `test_rv32im` 20
13 Processeur avec horloge et reset, programme `test_rv32im` : une instruction par période 22
14 Transcript ModelSim : processeur avec horloge, programmes `test_base`, `instr.txt` puis `instr2.txt` 22
15 Système final : processeur et bloc quadram 23
16 Écritures puis lecture dans le bloc `quadram` : la donnée lue n'est valide qu'après le front montant (curseurs à 140 et 145 ns) 25
17 Système complet avec le programme `instr2` 27
18 Les quatre écritures de `test_memoire` : le décodeur sélectionne successivement les quatre RAM 28

Liste des tableaux

1 Chronologie du test court de la RAM initiale . . . 4
2 Configurations vérifiées avec deux passes complémentaires . . . 6
3 Décodage des deux bits de sélection . . . 8
4 Répartition des adresses entre les quatre RAM de 256 mots . . . 9
5 Test des quatre RAM ; adresses et données en hexadécimal . . . 9
6 Configurations du quadram générique effectivement simulées . . . 11
7 Programme `instr.txt` . . . 13
8 Résultats de `instr.txt` avant et après correction du modèle . . . 13
9 Formats d'instructions utilisés (spécification RISC-V, chapitre 2.2) . . . 14
10 Codage de `add x9, x20, x21` . . . 14
11 Modes d'adressage . . . 15
12 Résultats de `test_base` (x2 = 100, x6 = 3 et x7 = 800 sont corrects dans les deux cas) . . . 16
13 Instructions de l'extension M (opcode `0110011`, funct7 = `0000001`) . . . 18
14 Résultats imposés par la spécification pour la division . . . 18
15 Résultats de `test_rv32im`, avec x1 = -7, x2 = 2, x11 = -2^{31} et x12 = -1 . . . 20
16 Ports ajoutés au processeur . . . 20
17 Accès à la mémoire pendant `test_base` sur le système . . . 25
18 Résultats de `test_memoire` sur le système . . . 28
19 Comparaison des deux programmes : seules les deux premières instructions changent . . . 29
20 Exécution des deux programmes avec mémoire interne puis externe . . . 29

Bibliographie

[1] Sébastien Pillement. *Mini-projet : Réutilisation de codes HDL et HDL réutilisable*. Sujet de mini-projet, ETN 5, Polytech Nantes. 2026.

[2] Andrew Waterman et Krste Asanović. *The RISC-V Instruction Set Manual, Volume I : Unprivileged ISA*. Document Version 20191213. RISC-V Foundation. 2019. URL : https://content.riscv.org/wp-content/uploads/2019/12/riscv-spec-20191213.pdf.

[3] *IEEE Standard for SystemVerilog : Unified Hardware Design, Specification, and Verification Language*. IEEE Std 1800-2017. IEEE. 2018. DOI : 10.1109/IEEESTD.2018.8299595.